

11-Other-AI-Standards-Summary

1 Other AI Standards Summary

Document Version: 1.0

Generated: December 4, 2025

Status: Reference Summary

1.1 Overview

This document provides concise summaries of additional AI/ML standards and formats beyond the core standards documented in detail elsewhere in this knowledge base. Each entry includes the standard's purpose, authoritative references, key tools, and pointers for further exploration.

1.2 Table of Contents

1. [Model Interchange Formats](#)
 2. [ML Lifecycle & Versioning](#)
 3. [Model Serving & Deployment](#)
 4. [Training & Optimization](#)
 5. [Data Management](#)
-

1.3 Model Interchange Formats

1.3.1 PMML (Predictive Model Markup Language)

Purpose: XML-based standard for representing statistical and data mining models.

Description: PMML enables sharing of predictive models between compliant applications. It supports classical ML algorithms (decision trees, regression, clustering, neural networks) and provides a vendor-neutral format primarily for traditional ML rather than deep learning.

Key Features: - XML-based format - Supports 20+ model types - Widely adopted in enterprise analytics - Strong support for classical ML algorithms - Limited deep learning support

Authoritative Reference: <https://dmg.org/pmml/v4-4-1/GeneralStructure.html>

Tools: - **KNIME:** Visual workflow tool with PMML export (<https://www.knime.com/>) - **R (pmml package):** Export R models to PMML (<https://cran.r-project.org/web/packages/pmml/>) - **SAS:** Native PMML support - **IBM SPSS:** PMML import/export - **jpmmml-evaluator:** Java runtime (<https://github.com/jpmml/jpmml-evaluator>)

Example Structure:

```

<?xml version="1.0"?>
<PMML version="4.4" xmlns="http://www.dmg.org/PMML-4_4">
  <Header copyright="Example Corp" description="Iris Classification Model"/>
  <DataDictionary numberOfFields="5">
    <DataField name="sepal_length" optype="continuous" dataType="double"/>
    <DataField name="species" optype="categorical" dataType="string">
      <Value value="setosa"/>
      <Value value="versicolor"/>
      <Value value="virginica"/>
    </DataField>
  </DataDictionary>
  <TreeModel modelName="IrisTree" functionName="classification">
    <MiningSchema>
      <MiningField name="sepal_length"/>
      <MiningField name="species" usageType="target"/>
    </MiningSchema>
    <Node score="setosa">
      <True/>
      <Node score="versicolor">
        <SimplePredicate field="sepal_length" operator="greaterThan" value="5.5"/>
      </Node>
    </Node>
  </TreeModel>
</PMML>

```

1.3.2 CoreML

Purpose: Apple's format for deploying ML models on iOS, macOS, watchOS, and tvOS devices.

Description: CoreML enables on-device machine learning inference with optimizations for Apple hardware (Neural Engine, GPU, CPU). Supports computer vision, natural language processing, and general-purpose models.

Key Features: - Optimized for Apple Silicon - On-device inference (privacy-friendly) - Supports quantization and optimization - Integration with iOS/macOS apps - Protobuf-based binary format

Authoritative Reference: <https://apple.github.io/coremltools/>

Tools: - **coremltools**: Python package for conversion (<https://github.com/apple/coremltools>) - **Xcode**: Native CoreML model integration -

Create ML: Apple's model training tool - **TensorFlow** → **CoreML**: Official converter - **PyTorch** → **CoreML**: Via coremltools

Conversion Example:

```

import coremltools as ct
import torch

# Convert PyTorch model
pytorch_model = torch.load("model.pth")
pytorch_model.eval()

# Trace model
example_input = torch.rand(1, 3, 224, 224)
traced_model = torch.jit.trace(pytorch_model, example_input)

# Convert to CoreML
coreml_model = ct.convert(
    traced_model,
    inputs=[ct.TensorType(shape=(1, 3, 224, 224))],
    outputs=[ct.TensorType(name="output")],
    minimum_deployment_target=ct.target.iOS15
)

# Save
coreml_model.save("ImageClassifier.mlmodel")

```

1.3.3 PFA (Portable Format for Analytics)

Purpose: JSON-based scoring engine for statistical models and data transformations.

Description: PFA provides a complete specification for analytic workflows including data preprocessing, model scoring, and postprocessing. Designed as a successor/alternative to PMML with better support for complex pipelines.

Key Features: - JSON-based (human-readable) - Turing-complete expression language - Supports model ensembles - Strong typing system - Production-ready scoring engine

Authoritative Reference: <http://dmg.org/pfa/>

Tools: - **Hadrian:** JVM-based PFA engine (<https://github.com/opendatagroup/hadrian>) - **Titus:** Python PFA engine (<https://github.com/opendatagroup/titus2>) - **Aurelius:** Python PFA producer (<https://github.com/opendatagroup/hadrian>)

Example:

```
{
  "name": "SimpleLinearRegression",
  "method": "map",
  "input": {"type": "record", "name": "Input", "fields": [
    {"name": "x", "type": "double"}
  ]},
  "output": "double",
  "action": [
    {"let": {"prediction": {"+": [
      {"*": [2.5, "input.x"]},
      1.3
    ]}}},
    "prediction"
  ]
}
```

1.3.4 NNEF (Neural Network Exchange Format)

Purpose: Khronos Group standard for exchanging neural network models.

Description: NNEF provides a simple, extensible format for representing trained neural networks. Focuses on inference deployment with hardware-agnostic representation.

Key Features: - Text-based graph representation - Binary tensor format - Extensible operator set - Hardware-neutral - Supports custom layers

Authoritative Reference: <https://www.khronos.org/nnef>

Tools: - **NNEF Tools:** Official parser and converter (<https://github.com/KhronosGroup/NNEF-Tools>) - **TensorFlow** → **NNEF:** Converter available - **ONNX** → **NNEF:** Conversion path

Example:

```
version 1.0;
graph G( input ) -> ( output ) {
  input = external(shape = [1, 3, 224, 224]);
  filter = variable(shape = [64, 3, 7, 7], label = 'conv1/weights');
  bias = variable(shape = [1, 64], label = 'conv1/bias');
  conv = conv(input, filter, bias, stride = [2, 2], padding = [(3,3), (3,3)]);
  relu = relu(conv);
  output = relu;
}
```

1.3.5 TorchScript

Purpose: PyTorch's model serialization format for production deployment.

Description: TorchScript creates serializable and optimizable models from PyTorch code. Enables deployment in non-Python environments and optimization for production.

Key Features: - Two modes: tracing and scripting - Optimized runtime - C++ deployment - Mobile support - Preserves model logic

Authoritative Reference: <https://pytorch.org/docs/stable/jit.html>

Tools: - **PyTorch:** Native support (<https://pytorch.org/>) - **TorchServe:** Serving with TorchScript (<https://pytorch.org/serve/>) - **LibTorch:** C++ inference library - **PyTorch Mobile:** iOS/Android deployment

Example:

```
import torch

class MyModule(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = torch.nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)

# Tracing
model = MyModule()
example_input = torch.rand(1, 10)
traced_model = torch.jit.trace(model, example_input)
traced_model.save("model_traced.pt")

# Scripting (preserves control flow)
scripted_model = torch.jit.script(model)
scripted_model.save("model_scripted.pt")

# Load and use
loaded = torch.jit.load("model_traced.pt")
output = loaded(torch.rand(1, 10))
```

1.4 ML Lifecycle & Versioning

1.4.1 DVC (Data Version Control)

Purpose: Git-like version control for ML models and datasets.

Description: DVC tracks large files, ML models, and datasets while keeping lightweight metafiles in Git. Enables reproducible ML pipelines and collaboration on ML projects.

Key Features: - Git-compatible workflow - Large file storage (S3, Azure, GCS, etc.) - Pipeline orchestration - Experiment tracking - Metrics versioning

Authoritative Reference: <https://dvc.org/doc>

Tools: - **DVC:** CLI tool (<https://dvc.org/>) - **DVC Studio:** Web UI for experiments (<https://studio.iterative.ai/>) - **CML:** CI/CD for ML (<https://cml.dev/>)

Example Workflow:

```
# Initialize DVC
dvc init

# Track data
dvc add data/training_set.csv
git add data/training_set.csv.dvc .gitignore
git commit -m "Add training data"

# Define pipeline
dvc run -n preprocess \
    -d data/raw.csv \
    -o data/processed.csv \
    python preprocess.py

dvc run -n train \
    -d data/processed.csv \
    -d train.py \
    -o model.pkl \
    -M metrics.json \
    python train.py

# Push data to remote storage
dvc push

# Reproduce pipeline
dvc repro
```

1.4.2 Kubeflow Pipelines

Purpose: Platform for building and deploying portable ML workflows on Kubernetes.

Description: Kubeflow Pipelines enables composition of ML workflows using Docker containers. Provides pipeline orchestration, experiment tracking, and execution history.

Key Features: - Kubernetes-native - Python SDK - Component reusability - Visualization - Experiment management

Authoritative Reference: <https://www.kubeflow.org/docs/components/pipelines/>

Tools: - **Kubeflow:** Full ML platform (<https://www.kubeflow.org/>) - **KFP SDK:** Python pipeline SDK (<https://kubeflow-pipelines.readthedocs.io/>) - **Argo Workflows:** Underlying orchestrator (<https://argoproj.github.io/workflows/>)

Example Pipeline:

```

from kfp import dsl, components

# Define component
@components.create_component_from_func
def train_model(data_path: str, model_path: components.OutputPath(str)):
    import joblib
    from sklearn.ensemble import RandomForestClassifier

    # Load data, train model
    model = RandomForestClassifier()
    # ... training code ...

    joblib.dump(model, model_path)

# Define pipeline
@dsl.pipeline(
    name='Training Pipeline',
    description='Train and deploy model'
)
def training_pipeline(data_path: str):
    train_task = train_model(data_path)
    deploy_task = deploy_model(train_task.outputs['model_path'])

# Compile
from kfp import compiler
compiler.Compiler().compile(training_pipeline, 'pipeline.yaml')

```

##Model Serving & Deployment

1.4.3 Seldon Core

Purpose: ML deployment platform for Kubernetes with advanced inference capabilities.

Description: Seldon Core provides production-grade ML model serving with support for A/B testing, canary deployments, explainability, and drift detection.

Key Features: - Kubernetes-native - Multi-framework support - Advanced deployment strategies - Explainability integration - Metrics and monitoring

Authoritative Reference: <https://docs.seldon.io/projects/seldon-core/>

Tools: - **Seldon Core:** OSS platform (<https://github.com/SeldonIO/seldon-core>) - **Seldon Deploy:** Enterprise platform (<https://www.seldon.io/products/deploy>) - **Alibi:** Explainability library (<https://github.com/SeldonIO/alibi>)

Example Deployment:

```

apiVersion: machinelearning.seldon.io/v1
kind: SeldonDeployment
metadata:
  name: iris-model
spec:
  predictors:
  - name: default
    graph:
      name: classifier
      implementation: SKLEARN_SERVER
      modelUri: s3://models/iris
    replicas: 2
    traffic: 100

```

1.4.4 BentoML

Purpose: Unified framework for building, shipping, and scaling ML services.

Description: BentoML simplifies ML model deployment by providing a standard format for packaging models with dependencies, API definitions, and serving infrastructure.

Key Features: - Framework-agnostic - Docker containerization - API server generation - Performance optimization - Cloud deployment

Authoritative Reference: <https://docs.bentoml.org/>

Tools: - **BentoML:** OSS framework (<https://github.com/bentoml/BentoML>) - **Yatai:** Deployment platform (<https://github.com/bentoml/Yatai>)

Example:

```
import bentoml
from bentoml.io import JSON, NumpyNdarray

# Save model
bentoml.sklearn.save_model("iris_classifier", model)

# Create service
@bentoml.service
class IrisClassifier:
    model = bentoml.sklearn.get("iris_classifier:latest")

    @bentoml.api
    def predict(self, input: NumpyNdarray) -> JSON:
        return self.model.predict(input)

# Build bento
bentoml.build()

# Containerize
bentoml.containerize("iris_classifier:latest")
```

1.4.5 Ray Serve

Purpose: Scalable model serving library built on Ray.

Description: Ray Serve enables scalable deployment of ML models with dynamic request batching, model composition, and multi-model serving on a distributed Ray cluster.

Key Features: - Built on Ray (distributed computing) - Framework-agnostic - Dynamic batching - Model composition - Autoscaling

Authoritative Reference: <https://docs.ray.io/en/latest/serve/>

Tools: - **Ray:** Distributed computing framework (<https://www.ray.io/>) - **Ray Serve:** Serving library (<https://docs.ray.io/en/latest/serve/>) - **Anyscale:** Managed Ray platform (<https://www.anyscale.com/>)

Example:

```

from ray import serve
import torch

@serve.deployment
class ImageClassifier:
    def __init__(self):
        self.model = torch.load("model.pt")

    async def __call__(self, request):
        image = await request.json()
        prediction = self.model(image)
        return {"class": prediction.argmax().item()}

# Deploy
serve.run(ImageClassifier.bind())

```

1.5 Training & Optimization

1.5.1 TensorRT

Purpose: NVIDIA's SDK for high-performance deep learning inference.

Description: TensorRT optimizes trained models for NVIDIA GPUs by applying graph optimizations, layer fusion, precision calibration (FP16, INT8), and kernel auto-tuning.

Key Features: - GPU-optimized inference - Multiple precision support (FP32, FP16, INT8) - Dynamic shapes - Plugin support for custom layers - C++ and Python APIs

Authoritative Reference: <https://developer.nvidia.com/tensorrt>

Tools: - **TensorRT:** Core SDK (<https://developer.nvidia.com/tensorrt>) - **TensorRT OSS:** Open-source components (<https://github.com/NVIDIA/TensorRT>) - **Triton Inference Server:** Uses TensorRT backend - **trt-exec:** Conversion utilities

Example:

```

import tensorrt as trt
import torch

# Export to ONNX first
torch.onnx.export(model, dummy_input, "model.onnx")

# Convert ONNX to TensorRT
logger = trt.Logger(trt.Logger.WARNING)
builder = trt.Builder(logger)
network = builder.create_network(1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH))
parser = trt.OnnxParser(network, logger)

with open("model.onnx", 'rb') as f:
    parser.parse(f.read())

config = builder.create_builder_config()
config.set_memory_pool_limit(trt.MemoryPoolType.WORKSPACE, 1 << 30) # 1GB
config.set_flag(trt.BuilderFlag.FP16) # Enable FP16

engine = builder.build_serialized_network(network, config)

# Save engine
with open("model.engine", 'wb') as f:
    f.write(engine)

```

1.5.2 OpenVINO

Purpose: Intel's toolkit for optimizing and deploying deep learning models.

Description: OpenVINO enables deployment of models across Intel hardware (CPUs, integrated GPUs, VPUs, FPGAs) with optimizations for edge and cloud deployments.

Key Features: - Intel hardware optimization - Model Optimizer for conversion - Inference Engine runtime - Pre-optimized model zoo - Edge deployment focus

Authoritative Reference: <https://docs.openvino.ai/>

Tools: - **OpenVINO:** Toolkit (<https://github.com/openvinotoolkit/openvino>) - **Model Optimizer:** Conversion tool - **Inference Engine:** Runtime - **Open Model Zoo:** Pre-trained models (https://github.com/openvinotoolkit/open_model_zoo)

Example:

```
# Convert TensorFlow model
mo --input_model model.pb \
   --output_dir openvino_model \
   --data_type FP16

# Run inference (Python)
from openvino.runtime import Core

ie = Core()
model = ie.read_model("model.xml")
compiled_model = ie.compile_model(model, "CPU")
output = compiled_model([input_data])
```

1.6 Data Management

1.6.1 Datasheets for Datasets

Purpose: Documentation framework for datasets (companion to Model Cards).

Description: Datasheets provide standardized documentation for datasets including motivation, composition, collection process, preprocessing, uses, distribution, and maintenance.

Key Sections: - Motivation - Composition - Collection Process - Preprocessing/Cleaning - Uses - Distribution - Maintenance

Authoritative Reference: <https://arxiv.org/abs/1803.09010>

Tools: - **Croissant:** Dataset metadata format (<https://github.com/mlcommons/croissant>) - **Hugging Face Datasets:** Integrated datasheets (<https://huggingface.co/docs/datasets/>)

Example Questions: - Why was the dataset created? - What do the instances represent? - How many instances are there? - Is there any missing information? - Does the dataset contain sensitive information? - Was any preprocessing performed?

1.6.2 Apache Arrow

Purpose: Cross-language development platform for in-memory data.

Description: Arrow provides a standardized columnar memory format for flat and hierarchical data, enabling zero-copy reads and efficient inter-process communication.

Key Features: - Language-agnostic format - Zero-copy reads - SIMD optimizations - Interprocess communication - Efficient serialization (Arrow IPC, Parquet)

Authoritative Reference: <https://arrow.apache.org/docs/>

Tools: - **PyArrow:** Python implementation (<https://arrow.apache.org/docs/python/>) - **Parquet:** Columnar storage format (<https://parquet.apache.org/>) - **Arrow Flight:** RPC framework (<https://arrow.apache.org/docs/format/Flight.html>)

Example:

```
import pyarrow as pa
import pyarrow.parquet as pq

# Create Arrow table
data = {
    'feature1': [1.0, 2.0, 3.0],
    'feature2': [4.0, 5.0, 6.0],
    'label': [0, 1, 0]
}
table = pa.table(data)

# Write to Parquet
pq.write_table(table, 'data.parquet')

# Read from Parquet
table = pq.read_table('data.parquet')

# Convert to pandas (zero-copy)
df = table.to_pandas()
```

1.7 Summary Table

Standard	Category	Format	Primary Use	Maturity
PMML	Interchange	XML	Classical ML	Mature
CoreML	Interchange	Protobuf	Apple devices	Mature
PFA	Interchange	JSON	Scoring pipelines	Stable
NEEF	Interchange	Text	NN exchange	Emerging
TorchScript	Interchange	Binary	PyTorch deployment	Mature
DVC	Lifecycle	Git-based	Data/model versioning	Mature
Kubeflow	Lifecycle	YAML	ML pipelines	Mature
Seldon Core	Serving	Kubernetes	Advanced serving	Mature
BentoML	Serving	Docker	Model packaging	Mature
Ray Serve	Serving	Python	Scalable serving	Emerging
TensorRT	Optimization	Binary	NVIDIA GPU	Mature
OpenVINO	Optimization	IR	Intel hardware	Mature
Datasheets	Governance	Markdown	Dataset docs	Emerging
Apache Arrow	Data	Binary	Data interchange	Mature

Navigation: [Back to Index](#) | [Previous: Triton Inference Server](#) | [Next: Vector Embeddings](#)